SE3092 | Platform Based Development

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY
Faculty of Computing | Department of Computer Science

Assignment 01

Designing a Personal Finance Management System for Android

A Problem-First, Scenario-Driven Engineering Challenge

Module Code	SE3092
Module Title	Platform Based Development
Assignment Type	**Group Assignment**
Contribution	40% of Final Grade
Release Date	11/04/2026
Submission Deadline	**24/05/2026**
Submission Mode	LMS Upload + Scheduled Live Demo & Presentation
Minimum SDK	API Level 26 (Android 8.0) Target SDK 34+
Technology Stack	Kotlin · Jetpack Compose · MVVM · Firebase
Prepared By	H.A.A.C Perera

1. Background — The Industry Philosophy

In production software engineering, clean requirements documents are a luxury that rarely exists. Engineers at high-performing organizations are handed context user research, friction logs, stakeholder narratives and are expected to derive clarity from ambiguity before writing a single line of code. The systems that genuinely change how people live are built by engineers who understand the problem behind the problem.

This assignment is an exercise in that discipline. Rather than a feature checklist, you are given a scenario a detailed, realistic account of a real person navigating a real financial problem. Your primary obligation is not implementation; it is comprehension and translation. You must read, analyses, model, and design before you open Android Studio. The quality of your thinking upstream will be more consequential than the elegance of any individual Composable.

2. User Persona

Before engaging with the scenario, study who Kavindu Silva is. This is not a fictional archetype invented for academic convenience. He is a composite drawn from documented interviews with junior engineers across the Sri Lankan technology industry. His frustrations are common. His failed attempts are representative.

PERSONA PROFILE	Mr. Kavindu Silva — The Ambitious Earner Without a System
Full Name	Kavindu Silva
Age	25 years old
Education	BSc (Hons) Software Engineering — graduated 12 months ago
Current Role	Junior Software Engineer, product-focused startup, Colombo
Monthly Salary	LKR 120,000 – 140,000 (variable: base + performance component)
Side Income	Freelance web projects (LKR 20,000–90,000/project). Google AdSense in USD. Cryptocurrency trading (USDT, ETH, altcoins)
Financial Goal	Purchase a MacBook Pro M4 approx. LKR 490,000.00 within 12 months
Current Savings	LKR 11,200.00 after 12 months of consistent earning
Personality	Driven and technically sharp. Impulsive with discretionary spending. Optimistic about income. avoidant about expenses.
Core Frustration	Cannot explain where his money goes. Every tracking attempt has failed within two weeks. Feels financially invisible despite earning well.

3. The Scenario — Kavindu's Financial Reality

3.1 The Setup

Kavindu Silva graduated 12 months ago and moved into his first full-time engineering role with the conviction that financial stability was finally within reach. The salary was transformative more consistent income than he had ever managed. He told himself he would save aggressively from month one. He did not. Twelve months later, with over LKR 1.6 million earned in aggregate, he has LKR 11,200.00 in his savings account. He cannot articulate where the rest went. He has genuinely tried to reconstruct it and failed.

3.2 The Income Problem

Kavindu's income is structurally irregular, and this is central to his difficulty. He earns from four distinct channels, each operating on different cycles, in different currencies, and with different levels of predictability:

- His monthly salary arrives on the 25th but fluctuates between LKR 120,000 and LKR 140,000 depending on performance bonuses. He rarely examines his payslip in detail and frequently cannot recall the exact amount deposited.
- He accepts freelance engagements primarily React and WordPress builds for local SMEs. Payments range from LKR 20,000 to LKR 90,000 per project, are milestone-based, and arrive irregularly. He has twice forgotten to follow up on outstanding invoices and once could not reconcile which payment corresponded to which project.
- His tech blog generates Google AdSense revenue paid monthly in USD, converted to LKR at the prevailing rate at the time of transfer. Some months yield the equivalent of LKR 3,500; others reach LKR 19,000. He checks the AdSense dashboard occasionally but never records what he actually receives.
- He actively trades USDT, ETH, and several altcoins on Binance. He recorded a significant gain in Q1 and a material loss in Q3. He does not maintain a running net position. When asked what his total crypto earnings are year-to-date, he cannot answer.

When Kavindu attempts to calculate his monthly income, he opens four separate applications, navigates to his bank statement, checks his email, and abandons the exercise before completing it. He estimates his monthly income at 'somewhere between LKR 160,000 and 210,000' but has never validated this figure.

3.3 The Expense Invisibility

Kavindu's expenditure is fragmented across channels he does not monitor in aggregate: bank card transactions, cash withdrawals, PickMe rides, UberEats orders, a gym membership auto-debited from a secondary account, and two subscription services he intended to cancel six months ago. His rent is LKR 34,000.00 though utilities fluctuate the effective monthly figure. Beyond committed costs, everything else is opaque. He once reviewed three months of bank statements and discovered he had spent LKR 24,000 on coffee and casual dining in a single month. The discovery was genuinely shocking to him.

He has no category system. He has no concept of what proportion of his income is discretionary versus committed. He has never seen a visual representation of how his spending is distributed. He is certain that such a view would be immediately clarifying and that he has simply never had a tool that produced it.

3.4 The Goal That Does Not Move

Kavindu intends to purchase a MacBook Pro M4. This is not a vanity purchase his current laptop throttles under compilation workloads and is unsuitable for the local development environment his freelance clients increasingly require. He has priced his preferred configuration at approximately LKR 490,000 from an authorised reseller.

Eight months ago, he created a Google Sheet titled 'MacBook Fund' with columns for monthly income, expenses, and savings. He populated it diligently for thirteen days. A freelance deadline consumed the following week. When he returned to the sheet three weeks later, the incomplete entries felt more discouraging than helpful and he did not open it again. He does not know what monthly savings rate is required to reach his target within twelve months, because he does not know what he currently saves. The goal is emotionally present and financially invisible.

3.5 The Failed Attempts

This is not Kavindu's first attempt to solve this problem. Over the past year, he has tried the following:

1. A Google Sheet — abandoned when manually entered formulas broke after row insertions, and the overhead of data entry exceeded his willingness to maintain it.
2. A Play Store budgeting application — uninstalled within nine days because it supported only USD, had no cloud synchronisation, and its information architecture assumed a single income source and a fixed monthly budget.
3. A notes application where he pasted bank transaction exports — functional for approximately four days before becoming an unreadable block of unlabelled figures.

4. His bank's native spending summary — which captured only card transactions, omitted cash and secondary-account activity entirely, and miscategorised a PickMe fare as 'online retail'.

Each failure had a proximate cause. The underlying cause was consistent: none of these tools were designed for someone with irregular multi-source income, impulsive discretionary spending, a specific quantified goal, and a low tolerance for friction in daily data entry. They were either too rigid, too manual, too narrow in their data model, or too unpleasant to open every day.

3.6 What a Solution Needs to Be

Kavindu has never written a requirements document. But a careful, patient conversation with him would surface in complaints, in hesitations, in wishful thinking — a coherent picture of what he actually needs. That picture is embedded in the pages above. Finding it, translating it into a technical specification, and building something that would genuinely change Kavindu's relationship with his money is your assignment.

SPECIAL NOTE

This scenario is your specification. There is no feature list. There are no pre-written user stories. There are no wireframes. Read it until you understand not just what Kavindu needs, but why every previous tool failed him and what a system designed specifically for someone like him would have to do differently.

4. Your Task — From Scenario to System

4.1 Phase One — Analysis

Your work begins before the IDE. You are expected to conduct a structured analysis of Kavindu's scenario. This analysis must be documented formally and will carry direct weight in the evaluation. The following questions are not instructions they are analytical lenses. Your answers, whether explicit in your documentation or implicit in your design decisions, will be assessed.

- Kavindu earns from four sources with different currencies, frequencies, and certainty levels. What must a system understand about income structure in order to give a user an accurate picture of what they actually earn monthly — not what they estimate?
- Kavindu has failed with four different tools. What do these failures share at a behavioural and UX level? What architectural or design decisions in your application would prevent the same outcome?
- Kavindu has a specific, quantified savings goal and a deadline. His previous tools never connected daily financial behaviour to that goal in a visible or motivating way. How should a well-designed system make goal progress feel real and financially grounded — not abstract?
- Given that your application cannot import bank data automatically, how should it handle expense entry in a way that minimises friction and maximises long-term user adherence? What must an expense record capture to be analytically useful at the end of a month?
- Kavindu has no concept of his discretionary-to-committed spending ratio. What visualisations or summary views would most efficiently reveal this to a user who does not think in financial categories? What is the minimum set of views that would give him meaningful financial self-awareness within the first week?

4.2 Phase Two — Design

Produce the following design artefacts, all of which must appear in your documentation submission:

- A structured requirements document distinguishing functional requirements, non-functional requirements, and constraints — derived directly from your reading of the scenario.
- A Firebase Firestore schema with justification for collection structure, indexing strategy, and multi-currency income handling.
- An MVVM architecture diagram mapping your screens, ViewModels, Repositories, and data sources.
- A minimum of three annotated wireframes or high-fidelity mockups covering the screens most critical to Kavindu's daily experience.

4.3 Phase Three — Implementation

Build the application. It will be evaluated not merely on technical correctness but on whether it would genuinely improve Kavindu's financial clarity. The following constraints apply without exception:

- The application must be written entirely in Kotlin. No Java interop in the application layer.
- All UI must be implemented with Jetpack Compose. No XML layouts will be accepted.
- MVVM architecture must be strictly observed. ViewModel logic must not reside in Composable functions. Data layer calls must not originate from the UI layer.
- Firebase Authentication must manage user identity. Hard-coded user data will receive a mark of zero for the Firebase criterion.
- Firebase Firestore must serve as the primary persistence layer. Firestore Security Rules must be submitted as part of your deliverables.
- The application must install and run correctly on Android API Level 26 (Android 8.0) and above, targeting API 34 or higher.
- All basic error states must be handled, no unhandled exceptions, no blank screens on network failure, no crashes on empty datasets.

5. Technical Requirements

Layer / Concern	Required Technology	Constraints & Notes
Language	Kotlin (100%)	No Java files in application layer. Kotlin DSL for Gradle.
UI Framework	Jetpack Compose	Material Design 3 components. No XML layouts accepted.
Navigation	Navigation Compose	Single-activity architecture. Typed arguments preferred.
Architecture Pattern	MVVM	Strict separation: UI → ViewModel → Repository → Model.
State Management	StateFlow / LiveData	No raw mutable state exposed from ViewModel to UI layer.
Async & Concurrency	Kotlin Coroutines	viewModelScope for all async operations.
Authentication	Firebase Authentication	Email/password mandatory. Google Sign-In is bonus credit.
Database	Firebase Firestore	Real-time listeners on dashboard. Security rules required.
Local Caching	Room (strongly recommended)	Offline persistence. Do not block UI on network unavailability.
Dependency Injection	Hilt (preferred) or Koin	No manual ViewModel instantiation inside Composable functions.
Minimum SDK	API Level 26 — Android 8.0 Oreo	Target SDK 34 or above. APK must install on Android 8.0+.

6. Deliverables

6.1 Source Code Repository

- Private GitHub or GitLab repository. Submit the access link via the LMS before the deadline.

- Commit history must reflect genuine, iterative development. A single-commit repository will be referred for academic integrity review.
- README.md must include: project overview, Firebase configuration steps, build instructions, and an architectural summary.

6.2 Technical Design Document

- Professionally formatted PDF — minimum 15 pages, no maximum.
- Must contain: requirements analysis, Firestore schema, MVVM diagram, annotated wireframes, and design decision justifications.
- Documentation that does not reference Kavindu's specific scenario or your specific implementation will be penalised.

6.3 APK File

- A release-signed or debug APK connecting to a live Firebase project. Local emulator configurations are not acceptable for submission.
- Validated on a physical device or correctly configured emulator running Android 8.0+.

6.4 Live Demo & Presentation

- A 15-minute scheduled session with the lecture panel during the demonstration week published on the LMS.
- You will run the application live, walk through the core user flows, and explain your architectural and schema decisions.
- The session is an engineering discussion, not a slide deck presentation. Prepare to answer direct questions about your ViewModel structure, Firestore queries, and requirement derivation process.

ACADEMIC INTEGRITY

Code sharing between students, submission of AI-generated code as original work without disclosure, and plagiarism of open-source finance applications will result in an automatic zero and referral to the Faculty Disciplinary Committee. AI tools may be used as a learning aid, but all code, design decisions, and documentation submitted must represent your own intellectual work.

7. Evaluation Criteria

The rubric below governs all assessment. Marks are weighted toward problem understanding, architecture, and demonstration not toward feature volume. A focused, well-reasoned application that solves Kavindu's core problem with architectural integrity will outscore a feature-dense application with structural weaknesses. Read this rubric before you begin your analysis.

#	Evaluation Criterion	What Is Being Assessed	Marks	Weight
1	**Problem Analysis & Requirement Extraction**	Quality and depth of the derived functional and non-functional requirements from the scenario. Evidence of independent thinking systems. Absence of generic feature listing. Peer reviews will be conducted, and the final marks will be determined based on the peer review outcomes.	20	20%
2	**Architecture & Code Quality**	Correct and clean MVVM implementation. Separation of UI, ViewModel, Repository, and Model. Use of Kotlin coroutines, StateFlow, dependency injection. No business logic in Composables.	20	20%
3	**Jetpack Compose UI & UX Design**	Composable decomposition quality. Navigation graph integrity. Adherence to Material Design 3. Extent to	15	15%

#	Evaluation Criterion	What Is Being Assessed	Marks	Weight
		which the design genuinely resolves Kavindu's usability failures — not merely functional screens.		
4	**Firebase Integration**	Firebase Authentication implementation. Firestore data modelling — collections, indexing, multi-currency handling. Real-time listeners. Offline persistence. Submitted security rules.	15	15%
5	**Feature Completeness & Innovation**	Core problem coverage. Handling of edge cases (irregular income, multi-source entries, goal progress). Meaningful enhancements beyond the minimum that genuinely improve the solution.	10	10%
6	**Technical Documentation**	Requirements document, Firestore schema, architecture diagram, annotated wireframes, README. Specificity to Kavindu's scenario — generic or templated documentation will be penalised.	10	10%
7	**Demo & Presentation**	Clarity and confidence in presenting the solution. Ability to explain architectural decisions, Firestore schema, and MVVM decomposition under questioning. APK runs live without errors.	10	10%
	TOTAL		**100**	**100%**

PARTIAL CREDIT	Partial credit is available within every criterion. A weak Firebase implementation scores partial marks rather than zero. However, absent components, no Firebase Authentication, no submitted security rules, an APK that fails to install, or no attendance at the demo session will receive zero for the relevant criterion without exception.

8. Special Note

The ability to navigate from that starting point to a shipped, functional product that a real user would actually open every day is not a skill that emerges from following instructions. It is a skill built through practice, through the discipline of thinking before building, and through honest self-assessment of whether what you have built actually solves the problem or merely resembles a solution.

Kavindu's problem is technically approachable. The architecture is well-documented. The Firebase APIs are mature. The challenge here is not the Kotlin or the Compose. it is understanding the problem with enough precision and empathy to know exactly what to build, and caring enough about the user experience to build it well.

If your submission makes us believe genuinely believe that Kavindu would open this application daily, that it would change how he understands his money, and that he would still be using it six months from now, you will have done this assignment justice. We look forward to your work.